

StickyMem

MVP Implementation Plan

Generated: 2026-05-06 | 12 Tasks | 22 Steps

Table of Contents

- Task 1: Project Initialization + Type Definitions
- Task 2: Local Database Layer
- Task 3: SM-2 Spaced Repetition Engine
- Task 4: DeepSeek API Client
- Task 5: LLM Content Digestion + Question Generation
- Task 6: Settings Screen — API Key Configuration
- Task 7: Add Content Screen
- Task 8: Review Session — Question + Feedback UI
- Task 9: Home Screen + Database Integration
- Task 10: Progress Screen + Chart
- Task 11: Notifications + Polish
- Task 12: Open Source Preparation

StickyMem MVP Implementation Plan

> For agentic workers: REQUIRED SUB-SKILL: Use subagent-driven-development (recommended) to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: Build a mobile-first React Native app where users paste text/URLs, the app automatically digests content and generates spaced repetition questions to help retain knowledge.

Architecture: Fully client-side Expo app. All data stored locally in SQLite. DeepSeek API calls made directly from device with user's own API key (BYOK). SM-2 algorithm runs locally for scheduling. Local push notifications for daily review reminders. No backend server required — zero infrastructure cost for MVP.

Tech Stack:

- Expo SDK 52+ with Expo Router (file-based routing)
- TypeScript throughout
- expo-sqlite (local database)
- expo-notifications (local push reminders)
- zustand (lightweight state management)
- DeepSeek API (chat/completions endpoint)
- react-native-chart-kit (progress charts)
- @expo/vector-icons (UI icons)

Task 1: Project Initialization + Type Definitions

Files:

- Create: package.json
- Create: app.json
- Create: tsconfig.json
- Create: .env.example
- Create: src/types/index.ts
- Create: app/_layout.tsx

☐ Step 1: Initialize Expo project

```
cd ~/projects/sticky-mem
npx create-expo-app@latest . --template blank-typescript
```

Expected: Project created with TypeScript template. Clean package.json, app.json, tsconfig.json.

☐ Step 2: Install MVP dependencies

```
npx expo install expo-router expo-sqlite expo-notifications expo-constants expo-linking expo-status-bar
npx expo install react-native-screens react-native-safe-area-context
npm install zustand react-native-chart-kit react-native-svg
npm install -D vitest @testing-library/react-native
```

Expected: All dependencies installed. Note: expo-router replaces the default App.js entry point — need to set "main": "expo-router/entry" in package.json.

☐ **Step 3: Configure Expo Router in package.json**

Edit package.json — add the "main" field and "scripts":

```
{
  "main": "expo-router/entry",
  "scripts": {
    "start": "expo start",
    "android": "expo start --android",
    "ios": "expo start --ios",
    "web": "expo start --web",
    "test": "vitest",
    "test:watch": "vitest --watch"
  }
}
```

☐ **Step 4: Write .env.example**

```
# StickyMem — Bring Your Own Key
# Get your DeepSeek API key at https://platform.deepseek.com/
DEEPSEEK_API_KEY=sk-your-key-here
```

☐ **Step 5: Write TypeScript type definitions**

File: src/types/index.ts

```

// — Content Types — //

export type ContentSource = 'text' | 'url';

export interface Content {
  id: string;           // UUID
  sourceType: ContentSource;
  title: string;        // User-provided or auto-generated
  rawText: string;      // Original input text
  createdAt: number;    // Unix timestamp ms
  updatedAt: number;    // Unix timestamp ms
}

// — Question Types — //

export type QuestionType = 'multiple_choice' | 'short_answer';
export type ReviewGrade = 0 | 1 | 2 | 3 | 4 | 5; // SM-2 grade

export interface Question {
  id: string;           // UUID
  contentId: string;    // FK to Content
  type: QuestionType;
  question: string;     // The question text
  correctAnswer: string;
  options?: string[];   // For multiple_choice (4 options, shuffled)
  explanation?: string; // Shown after answering
  createdAt: number;
}

// — Review / SM-2 Types — //

export interface ReviewRecord {
  id: string;
  questionId: string;
  gradedAt: number;
  grade: ReviewGrade;
}

export interface Card {
  // SM-2 per-card scheduling state persisted in DB
  questionId: string;
  easiness: number;      // Default 2.5
  interval: number;      // Days until next review
  repetitions: number;   // Consecutive correct answers
  nextReviewAt: number;  // Unix timestamp ms
  lastReviewAt: number;  // Unix timestamp ms or 0
}

// — Aggregated Types — //

export interface ReviewSession {
  dueCards: (Question & { card: Card })[];
  currentIndex: number;
  isComplete: boolean;
}

export interface DailyStats {
  date: string;          // YYYY-MM-DD
  totalReviewed: number;
  correctCount: number;
  accuracy: number;      // 0.0 - 1.0
}

```

☐ **Step 6: Write root layout with providers**

File: app/_layout.tsx

```
import { Stack } from 'expo-router';
import { StatusBar } from 'expo-status-bar';

export default function RootLayout() {
  return (
    <>
      <StatusBar style="dark" />
      <Stack
        screenOptions={{
          headerStyle: { backgroundColor: 'fff' },
          headerTintColor: '#333',
          headerTitleStyle: { fontWeight: 'bold' },
        }}
      >
        <Stack.Screen name="index" options={{ title: 'Today' }} />
        <Stack.Screen name="add" options={{ title: 'Add Content', presentation: 'modal' }} />
        <Stack.Screen name="review" options={{ title: 'Review', headerBackVisible: false }} />
        <Stack.Screen name="progress" options={{ title: 'Progress' }} />
        <Stack.Screen name="settings" options={{ title: 'Settings' }} />
      </Stack>
    </>
  );
}
```

☐ **Step 7: Commit**

```
git init
git add -A
git commit -m "chore: initial project setup with Expo Router + TypeScript types"
```

Task 3: SM-2 Spaced Repetition Engine

Files:

- Create: src/engine/sm2.ts
- Create: tests/sm2.test.ts

☐ **Step 1: Write the failing tests for SM-2**

File: tests/sm2.test.ts

```

import { describe, it, expect } from 'vitest';
import { sm2, calculateNextReview } from '../src/engine/sm2';

describe('SM-2 Algorithm', () => {
  it('should return easiness >= 1.3 after any grade', () => {
    const result = sm2({ easiness: 2.5, interval: 1, repetitions: 0 }, 0);
    expect(result.easiness).toBeGreaterThanOrEqual(1.3);
  });

  it('should reset repetitions on grade < 3', () => {
    const result = sm2({ easiness: 2.5, interval: 7, repetitions: 5 }, 0);
    expect(result.repetitions).toBe(0);
    expect(result.interval).toBe(1);
  });

  it('should set interval to 1 on first correct answer', () => {
    const result = sm2({ easiness: 2.5, interval: 0, repetitions: 0 }, 4);
    expect(result.interval).toBe(1);
    expect(result.repetitions).toBe(1);
  });

  it('should set interval to 6 on second correct answer', () => {
    const result = sm2({ easiness: 2.5, interval: 1, repetitions: 1 }, 4);
    expect(result.interval).toBe(6);
    expect(result.repetitions).toBe(2);
  });

  it('should multiply interval by easiness on 3rd+ correct', () => {
    // After 2 correct: interval=6, easiness=2.5
    // Next: 6 * 2.5 = 15
    const result = sm2({ easiness: 2.5, interval: 6, repetitions: 2 }, 4);
    expect(result.interval).toBe(15);
    expect(result.repetitions).toBe(3);
  });

  it('should handle a perfect grade (5) correctly', () => {
    const result = sm2({ easiness: 2.5, interval: 6, repetitions: 2 }, 5);
    // easiness = 2.5 + 0.1 = 2.6
    expect(result.easiness).toBeCloseTo(2.6, 1);
    expect(result.interval).toBe(15); // 6 * 2.6 = 15.6 -> floor 15
  });

  it('should decrease easiness on poor grade', () => {
    const result = sm2({ easiness: 2.5, interval: 6, repetitions: 2 }, 1);
    // easiness = 2.5 - 0.8 = 1.7
    expect(result.easiness).toBeCloseTo(1.7, 1);
    expect(result.repetitions).toBe(0);
  });
});

describe('calculateNextReview', () => {
  it('should return a timestamp in the future', () => {
    const now = Date.now();
    const next = calculateNextReview(1, now);
    expect(next).toBeGreaterThan(now);
  });

  it('should return exactly interval days later', () => {
    const now = 1000000;
    const next = calculateNextReview(7, now);
    expect(next).toBe(1000000 + 7 * 24 * 60 * 60 * 1000);
  });
});

```



```
it('should return 0 for interval 0', () => {
```



```
expect(calculateNextReview(0, Date.now())).toBe(0);
```

```
});
```

```
});
```

☐ **Step 2: Run tests to verify they fail**

```
cd ~/projects/sticky-mem  
npx vitest run tests/sm2.test.ts
```

Expected: FAIL — module not found for ../src/engine/sm2.

☐ **Step 3: Write SM-2 algorithm implementation**

File: src/engine/sm2.ts

```

export interface SM2Input {
  easiness: number;
  interval: number;    // days
  repetitions: number;
}

export interface SM2Output {
  easiness: number;
  interval: number;    // days
  repetitions: number;
}

/**
 * SM-2 algorithm by Piotr Woźniak.
 *
 * Grade scale:
 * 5 – perfect response
 * 4 – correct after hesitation
 * 3 – correct with serious difficulty
 * 2 – incorrect; correct answer seemed easy to recall
 * 1 – incorrect; correct answer remembered upon seeing it
 * 0 – complete blackout
 *
 * Grade >= 3 is considered "correct" for interval progression.
 */
export function sm2(card: SM2Input, grade: number): SM2Output {
  let { easiness, interval, repetitions } = card;

  // Update easiness factor
  easiness = easiness + (0.1 - (5 - grade) * (0.08 + (5 - grade) * 0.02));
  if (easiness < 1.3) easiness = 1.3;

  if (grade < 3) {
    // Incorrect – reset
    repetitions = 0;
    interval = 1;
  } else {
    // Correct – advance interval
    repetitions += 1;
    if (repetitions === 1) {
      interval = 1;
    } else if (repetitions === 2) {
      interval = 6;
    } else {
      interval = Math.round(interval * easiness);
    }
  }

  return { easiness, interval, repetitions };
}

/**
 * Calculate the next review timestamp from an interval (in days).
 */
export function calculateNextReview(intervalDays: number, now: number): number {
  if (intervalDays <= 0) return 0;
  return now + intervalDays * 24 * 60 * 60 * 1000;
}

```

☐ Step 4: Run tests to verify they pass

```
cd ~/projects/sticky-mem
npx vitest run tests/sm2.test.ts
```

Expected: PASS — all 8 tests pass.

☐ **Step 5: Commit**

```
git add src/engine/ tests/sm2.test.ts
git commit -m "feat: add SM-2 spaced repetition algorithm"
```

Task 5: LLM Content Digestion + Question Generation

Files:

- Create: src/llm/digest.ts
- Create: src/llm/questions.ts
- Create: tests/digest.test.ts
- Create: tests/questions.test.ts

☐ **Step 1: Write the failing tests for digest**

File: tests/digest.test.ts

```

import { describe, it, expect, vi, beforeEach } from 'vitest';
import { digestContent } from '../src/llm/digest';

describe('digestContent', () => {
  const mockClient = {
    chat: vi.fn(),
  };

  beforeEach(() => {
    vi.restoreAllMocks();
  });

  it('should return key concepts from text', async () => {
    mockClient.chat.mockResolvedValue(JSON.stringify({
      title: 'Introduction to ETFs',
      keyConcepts: [
        'ETF is a basket of securities traded on exchanges',
        'Expense ratio is the annual fee',
        'Dividend yield varies by fund',
      ],
    }));

    const result = await digestContent(mockClient, 'ETF stands for exchange-traded fund...');
    expect(result.title).toBe('Introduction to ETFs');
    expect(result.keyConcepts).toHaveLength(3);
  });

  it('should parse URL from input and extract title', async () => {
    mockClient.chat.mockResolvedValue(JSON.stringify({
      title: 'Understanding Neural Networks',
      keyConcepts: ['Neurons are basic units', 'Weights determine signal strength'],
    }));

    const result = await digestContent(mockClient, 'Check this: https://example.com/ai');
    expect(result.title).toContain('Neural');
  });

  it('should handle empty input gracefully', async () => {
    await expect(digestContent(mockClient, '')).rejects.toThrow('Content is empty');
  });

  it('should handle malformed LLM response', async () => {
    mockClient.chat.mockResolvedValue('not json at all');
    await expect(digestContent(mockClient, 'Some text')).rejects.toThrow('Failed to parse');
  });
});

```

□ Step 2: Write the failing tests for questions

File: tests/questions.test.ts

```

import { describe, it, expect, vi, beforeEach } from 'vitest';
import { generateQuestions } from '../src/llm/questions';

describe('generateQuestions', () => {
  const mockClient = {
    chat: vi.fn(),
  };

  beforeEach(() => {
    vi.restoreAllMocks();
  });

  it('should generate questions from key concepts', async () => {
    mockClient.chat.mockResolvedValue(JSON.stringify([
      {
        type: 'multiple_choice',
        question: 'What is an ETF?',
        correctAnswer: 'A basket of securities',
        options: ['A basket of securities', 'A single stock', 'A bond', 'A currency'],
        explanation: 'ETF pools multiple assets into one fund.',
      },
      {
        type: 'short_answer',
        question: 'What determines signal strength in a neural network?',
        correctAnswer: 'Weights',
        explanation: 'Weights are parameters that amplify or dampen signals.',
      },
    ]));

    const result = await generateQuestions(mockClient, [
      'ETF is a basket of securities',
      'Weights determine signal strength',
    ], 'Finance');

    expect(result).toHaveLength(2);
    expect(result[0].type).toBe('multiple_choice');
    expect(result[0].options).toHaveLength(4);
    expect(result[1].type).toBe('short_answer');
  });

  it('should handle LLM returning invalid JSON', async () => {
    mockClient.chat.mockResolvedValue('```json\n{"invalid": true}\n```');
    await expect(generateQuestions(mockClient, ['test'], 'General'))
      .rejects.toThrow('Expected array');
  });

  it('should validate each question has required fields', async () => {
    mockClient.chat.mockResolvedValue(JSON.stringify([
      { type: 'multiple_choice', question: 'Q1' }, // missing correctAnswer
    ]));
    await expect(generateQuestions(mockClient, ['test'], 'General'))
      .rejects.toThrow('Invalid question');
  });
});

```

❑ Step 3: Run tests to verify they fail

```

cd ~/projects/sticky-mem
npx vitest run tests/digest.test.ts tests/questions.test.ts

```

Expected: FAIL — modules not found.

□ Step 4: Write content digestion module

File: src/llm/digest.ts

```
import type { DeepseekClient } from '../api/deepseek';

export interface DigestResult {
  title: string;
  keyConcepts: string[];
}

const DIGEST_SYSTEM_PROMPT = `You are a learning assistant that extracts key concepts from text.

Given any text or URL content, output a JSON object with:
1. "title": A short, descriptive title (max 10 words)
2. "keyConcepts": An array of 3-8 concise, standalone fact statements about the most important concepts

Rules:
- Each key concept should be a complete sentence
- Concepts must be self-contained (understandable without context)
- Focus on memorable, testable facts
- Remove fluff and examples
- Output valid JSON only, no markdown formatting`;

export async function digestContent(
  client: DeepseekClient,
  input: string,
): Promise<DigestResult> {
  if (!input || input.trim() === '') {
    throw new Error('Content is empty');
  }

  const userMessage = `Extract key concepts from this content:\n\n${input}`;

  const response = await client.chat(
    [{ role: 'user', content: userMessage }],
    {
      system: DIGEST_SYSTEM_PROMPT,
      temperature: 0.3,
      maxTokens: 2048,
    },
  );

  try {
    const parsed = JSON.parse(response);
    if (!parsed.title || !Array.isArray(parsed.keyConcepts)) {
      throw new Error('Invalid digest format');
    }
    return {
      title: parsed.title,
      keyConcepts: parsed.keyConcepts.slice(0, 8),
    };
  } catch (e) {
    throw new Error(`Failed to parse LLM response: ${e instanceof Error ? e.message : String(e)}`);
  }
}
```

□ Step 5: Write question generation module

File: src/llm/questions.ts

```

import type { DeepseekClient } from '../api/deepseek';
import type { Question, QuestionType } from '../types';

export interface QuestionInput {
  type: QuestionType;
  question: string;
  correctAnswer: string;
  options?: string[];
  explanation?: string;
}

const QUESTIONS_SYSTEM_PROMPT = `You are a quiz generator. Given key concepts and a topic, generate 3-6 que...

Output a JSON array of question objects. Each object must have:
- "type": "multiple_choice" or "short_answer"
- "question": The question text
- "correctAnswer": The correct answer (string)
- "options": [4 strings] – ONLY for multiple_choice, include correctAnswer as one of them
- "explanation": Brief explanation of the correct answer

Rules:
- Mix question types (at least 1 of each)
- Questions should test understanding, not trivia
- Wrong options should be plausible
- Output valid JSON array only, no markdown`;

export async function generateQuestions(
  client: DeepseekClient,
  keyConcepts: string[],
  topic: string,
): Promise<QuestionInput[]> {
  const userMessage = `Topic: ${topic}\n\nKey Concepts:\n${keyConcepts.map((c, i) => `${i + 1}. ${c}`).join...

  const response = await client.chat(
    [{ role: 'user', content: userMessage }],
    {
      system: QUESTIONS_SYSTEM_PROMPT,
      temperature: 0.5,
      maxTokens: 4096,
    },
  );

  const cleaned = response.replace(/```json\s*/g, '').replace(/```\s*/g, '').trim();
  let parsed: any[];
  try {
    parsed = JSON.parse(cleaned);
  } catch {
    throw new Error('Failed to parse LLM response as JSON array');
  }

  if (!Array.isArray(parsed)) {
    throw new Error('Expected array of questions');
  }

  // Validate each question
  for (const q of parsed) {
    if (!q.type || !q.question || !q.correctAnswer) {
      throw new Error('Invalid question: missing required fields');
    }
    if (q.type === 'multiple_choice' && (!q.options || q.options.length !== 4)) {
      throw new Error('Multiple choice question must have exactly 4 options');
    }
  }
}

```

}


```
return parsed;
```

}

☐ **Step 6: Run tests to verify they pass**

```
cd ~/projects/sticky-mem
npx vitest run tests/digest.test.ts tests/questions.test.ts
```

Expected: PASS — all 7 tests pass.

☐ **Step 7: Commit**

```
git add src/llm/ tests/digest.test.ts tests/questions.test.ts
git commit -m "feat: add LLM content digestion + question generation"
```

Task 7: Add Content Screen

Files:

- Create: app/add.tsx
- Create: src/components/AddContentForm.tsx

☐ **Step 1: Write AddContentForm component**

File: src/components/AddContentForm.tsx

```

import React, { useState } from 'react';
import { View, TextInput, TouchableOpacity, Text, StyleSheet, ActivityIndicator } from 'react-native';

interface Props {
  onSubmit: (input: string) => Promise<void>;
  isProcessing: boolean;
}

export default function AddContentForm({ onSubmit, isProcessing }: Props) {
  const [input, setInput] = useState('');
  const [mode, setMode] = useState<'text' | 'url'>('text');

  const handleSubmit = async () => {
    if (!input.trim() || isProcessing) return;
    await onSubmit(input.trim());
    setInput('');
  };

  return (
    <View style={styles.container}>
      <View style={styles.tabRow}>
        <TouchableOpacity
          style={[styles.tab, mode === 'text' && styles.tabActive]}
          onPress={() => setMode('text')}
        >
          <Text style={[styles.tabText, mode === 'text' && styles.tabTextActive]}>Paste Text</Text>
        </TouchableOpacity>
        <TouchableOpacity
          style={[styles.tab, mode === 'url' && styles.tabActive]}
          onPress={() => setMode('url')}
        >
          <Text style={[styles.tabText, mode === 'url' && styles.tabTextActive]}>URL</Text>
        </TouchableOpacity>
      </View>

      {mode === 'text' ? (
        <TextInput
          style={styles.textArea}
          value={input}
          onChangeText={setInput}
          placeholder="Paste the content you want to remember..."
          multiline
          textAlignVertical="top"
        />
      ) : (
        <TextInput
          style={styles.urlInput}
          value={input}
          onChangeText={setInput}
          placeholder="https://example.com/article"
          autoCapitalize="none"
          autoCorrect={false}
          keyboardType="url"
        />
      )}

      <TouchableOpacity
        style={[styles.submitButton, (!input.trim() || isProcessing) && styles.submitButtonDisabled]}
        onPress={handleSubmit}
        disabled={!input.trim() || isProcessing}
      >
        {isProcessing ? (

```



```
<ActivityIndicator color="#fff" />
```

) : (

<Text style={styles.submitText}>Digest & Generate Questions</Text>

}}

</TouchableOpacity>

</View>

);

}


```
const styles = StyleSheet.create({
```

```
container: { padding: 16 },
```

```
tabRow: { flexDirection: 'row', marginBottom: 16, gap: 8 },
```

```
tab: {
```

`flex: 1, paddingVertical: 10, borderRadius: 8,`

```
backgroundColor: '#F0F0F0', alignItems: 'center',
```

},


```
tabActive: { backgroundColor: '#4A90D9' },
```

```
tabText: { fontSize: 14, fontWeight: '500', color: '#666' },
```

```
tabTextActive: { color: '#fff' },
```

```
textArea: {
```

```
borderWidth: 1, borderColor: '#DDD', borderRadius: 10,
```

padding: 14, fontSize: 16, backgroundColor: '#F9F9F9',

```
minHeight: 200, textAlignVertical: 'top',
```

},


```
urlInput: {
```

```
borderWidth: 1, borderColor: '#DDD', borderRadius: 10,
```

padding: 14, fontSize: 16, backgroundColor: '#F9F9F9',

},

```
submitButton: {
```

```
backgroundColor: '#4A90D9', borderRadius: 10, padding: 16,
```

```
alignItems: 'center', marginTop: 20,
```

},


```
submitButtonDisabled: { backgroundColor: '#CCC' },
```

```
submitText: { color: '#fff', fontSize: 16, fontWeight: '600' },
```

```
});
```

□ Step 2: Write Add Content screen

File: app/add.tsx

```
import React, { useState } from 'react';
import { View, Alert, StyleSheet } from 'react-native';
import { useRouter } from 'expo-router';
import AddContentForm from '../src/components/AddContentForm';
import { useApiClient } from '../src/hooks/useApiClient';
import { digestContent } from '../src/llm/digest';
import { generateQuestions } from '../src/llm/questions';
import { Repository } from '../src/db/repository';
import { useSettingsStore } from '../src/store/settings';
import { useDatabase } from '../src/hooks/useDatabase';

export default function AddContentScreen() {
  const [isProcessing, setIsProcessing] = useState(false);
  const apiClient = useApiClient();
  const router = useRouter();

  // Note: useDatabase hook will be created in a later step
  // For now, this is a placeholder – the actual DB init happens in Task 9

  const handleSubmit = async (input: string) => {
    if (!apiClient) {
      Alert.alert('API Key Required', 'Please configure your DeepSeek API key in Settings first.');
```

router.push('/settings');

return;

}

setIsProcessing(true);

try {

 const digest = await digestContent(apiClient, input);

 const questions = await generateQuestions(apiClient, digest.keyConcepts, digest.title);

 Alert.alert(

 'Success!',

 `Digested "\${digest.title}"\nGenerated \${questions.length} questions.\nThey will appear in your rev...`,

 [{ text: 'Great!', onPress: () => router.back() }],

);

} catch (error) {

 Alert.alert(

 'Error',

 `Failed to process content: \${error instanceof Error ? error.message : 'Unknown error'}`,

);

} finally {

 setIsProcessing(false);

}

};

return (

 <View style={styles.container}>

 <AddContentForm onSubmit={handleSubmit} isProcessing={isProcessing} />

 </View>

);

}

const styles = StyleSheet.create({

 container: { flex: 1, backgroundColor: '#fff' },

});

□ Step 3: Commit

```
git add app/add.tsx src/components/AddContentForm.tsx
git commit -m "feat: add content input screen with text/URL modes"
```

Task 9: Home Screen + Database Integration

Files:

- Create: src/hooks/useDatabase.ts
- Create: app/index.tsx (Home screen)
- Create: src/hooks/useSchedule.ts
- Modify: app/add.tsx (wire up database persistence)
- Modify: app/review.tsx (load due cards from DB)

□ Step 1: Write useDatabase hook

File: src/hooks/useDatabase.ts

```
import { useEffect, useState, useRef } from 'react';
import * as SQLite from 'expo-sqlite';
import { Repository } from '../db/repository';

let globalDb: SQLite.SQLiteDatabase | null = null;
let globalRepo: Repository | null = null;

export async function initDatabase(): Promise<Repository> {
  if (globalRepo) return globalRepo;
  globalDb = await SQLite.openDatabaseAsync('stickymem.db');
  globalRepo = await Repository.create({
    run: (sql: string, params?: any[]) => globalDb!.runAsync(sql, params),
    query: <T,>(sql: string, params?: any[]) => globalDb!.getAllAsync(sql, params) as Promise<T[]>,
  });
  return globalRepo;
}

export function useDatabase(): Repository | null {
  const [repo, setRepo] = useState<Repository | null>(null);
  const initialized = useRef(false);

  useEffect(() => {
    if (initialized.current) return;
    initialized.current = true;
    initDatabase().then(setRepo).catch(console.error);
  }, []);

  return repo;
}
```

□ Step 2: Write useSchedule hook

File: src/hooks/useSchedule.ts

```

import { useState, useEffect } from 'react';
import type { Repository } from '../db/repository';
import type { Question, Card } from '../types';

export function useSchedule(repo: Repository | null) {
  const [dueCount, setDueCount] = useState(0);
  const [totalQuestions, setTotalQuestions] = useState(0);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    if (!repo) return;
    loadStats();

    async function loadStats() {
      try {
        const due = await repo.getDueCards(Date.now());
        const total = await repo.getTotalQuestionCount();
        setDueCount(due.length);
        setTotalQuestions(total);
      } catch (e) {
        console.error('Failed to load schedule:', e);
      } finally {
        setLoading(false);
      }
    }
  }, [repo]);

  const loadDueCards = async (): Promise<(Question & { card: Card })[]> => {
    if (!repo) return [];
    return repo.getDueCards(Date.now());
  };

  return { dueCount, totalQuestions, loading, loadDueCards };
}

```

□ Step 3: Write Home screen

File: app/index.tsx

```

import React from 'react';
import { View, Text, TouchableOpacity, StyleSheet, ScrollView } from 'react-native';
import { useRouter } from 'expo-router';
import { useDatabase } from '../src/hooks/useDatabase';
import { useSchedule } from '../src/hooks/useSchedule';
import { useSettingsStore } from '../src/store/settings';
import { Ionicons } from '@expo/vector-icons';

export default function HomeScreen() {
  const router = useRouter();
  const repo = useDatabase();
  const { dueCount, totalQuestions, loading } = useSchedule(repo);
  const isConfigured = useSettingsStore((s) => s.isConfigured);

  return (
    <View style={styles.container}>
      <ScrollView contentContainerStyle={styles.content}>
        { /* Header */ }
        <View style={styles.header}>
          <Text style={styles.appName}>StickyMem</Text>
          <TouchableOpacity onPress={() => router.push('/settings')}>
            <Ionicons name="settings-outline" size={24} color="#666" />
          </TouchableOpacity>
        </View>

        { /* Today's Review Card */ }
        <TouchableOpacity
          style={styles.reviewCard}
          onPress={() => router.push('/review')}
          disabled={dueCount === 0}
        >
          <Text style={styles.reviewCardTitle}>Today's Review</Text>
          {loading ? (
            <Text style={styles.reviewCardCount}>Loading...</Text>
          ) : dueCount > 0 ? (
            <>
              <Text style={styles.reviewCardCount}>{dueCount}</Text>
              <Text style={styles.reviewCardLabel}>questions due</Text>
              <TouchableOpacity style={styles.startButton}>
                <Text style={styles.startButtonText}>Start Review</Text>
              </TouchableOpacity>
            </>
          ) : totalQuestions > 0 ? (
            <>
              <Text style={styles.reviewCardCount}>0</Text>
              <Text style={styles.reviewCardLabel}>questions due – you're all caught up! </Text>
            </>
          ) : (
            <>
              <Text style={styles.reviewCardCount}>0</Text>
              <Text style={styles.reviewCardLabel}>No content yet. Add something to learn!</Text>
            </>
          )}
        </TouchableOpacity>

        { /* Stats Row */ }
        <View style={styles.statsRow}>
          <View style={styles.statBox}>
            <Text style={styles.statNumber}>{totalQuestions}</Text>
            <Text style={styles.statLabel}>Total Questions</Text>
          </View>
          <View style={styles.statBox}>

```

```
<Text style={styles.statNumber}>{dueCount}</Text>
```



```
<Text style={styles.statLabel}>Due Today</Text>
```

</View>

</View>


```
{/* API Key Warning */}
```

```
{!isConfigured && (
```

```
<TouchableOpacity style={styles.warningCard} onPress={() => router.push('/settings')}>
```

```
<Ionicons name="warning-outline" size={20} color="#E65100" />
```



```
<Text style={styles.warningText}>
```

API key not configured. Go to Settings to add your DeepSeek key.

</Text>

</TouchableOpacity>

}}


```
{/* Quick Actions */}
```

```
<View style={styles.actionsRow}>
```



```
<TouchableOpacity style={styles.actionButton} onPress={() => router.push('/add')}>
```

```
<Ionicons name="add-circle-outline" size={28} color="#4A90D9" />
```

```
<Text style={styles.actionText}>Add Content</Text>
```

</TouchableOpacity>

```
<TouchableOpacity style={styles.actionButton} onPress={() => router.push('/progress')}>
```

```
<Ionicons name="trending-up-outline" size={28} color="#4A90D9" />
```

```
<Text style={styles.actionText}>Progress</Text>
```

</TouchableOpacity>

</View>

</ScrollView>

</View>

);

}


```
const styles = StyleSheet.create({
```

```
container: { flex: 1, backgroundColor: '#F5F5F5' },
```



```
content: { padding: 20 },
```

```
header: {
```

```
flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center',
```

marginBottom: 24, marginTop: 8,

},

```
appName: { fontSize: 28, fontWeight: '800', color: '#333' },
```

```
reviewCard: {
```

```
backgroundColor: '#fff', borderRadius: 20, padding: 24,
```



```
alignItems: 'center', marginBottom: 20,
```

```
shadowColor: '#000', shadowOffset: { width: 0, height: 4 },
```

shadowOpacity: 0.1, shadowRadius: 12, elevation: 4,

},

```
reviewCardTitle: { fontSize: 18, fontWeight: '600', color: '#666', marginBottom: 8 },
```

```
reviewCardCount: { fontSize: 64, fontWeight: '800', color: '#4A90D9' },
```

```
reviewCardLabel: { fontSize: 14, color: '#888', marginTop: 4, marginBottom: 16 },
```

```
startButton: {
```



```
backgroundColor: '#4A90D9', borderRadius: 12, paddingHorizontal: 32, paddingVertical: 12,
```

},

```
startButtonText: { color: '#fff', fontSize: 16, fontWeight: '600' },
```

```
statsRow: { flexDirection: 'row', gap: 12, marginBottom: 20 },
```

```
statBox: {
```

```
flex: 1, backgroundColor: '#fff', borderRadius: 14, padding: 16, alignItems: 'center',
```

},

```
statNumber: { fontSize: 32, fontWeight: '800', color: '#333' },
```



```
statLabel: { fontSize: 12, color: '#888', marginTop: 4 },
```

```
warningCard: {
```

```
flexDirection: 'row', backgroundColor: '#FFF3E0', borderRadius: 12, padding: 14,
```

```
alignItems: 'center', gap: 10, marginBottom: 20,
```

},

```
warningText: { flex: 1, fontSize: 13, color: '#E65100', lineHeight: 18 },
```

```
actionsRow: { flexDirection: 'row', gap: 12 },
```

```
actionButton: {
```


flex: 1, backgroundColor: '#fff', borderRadius: 14, padding: 20,

```
alignItems: 'center', gap: 8,
```

},

```
actionText: { fontSize: 14, fontWeight: '500', color: '#666' },
```

```
});
```

□ **Step 4: Update add.tsx to persist to database**

Update app/add.tsx — add imports and modify handleSubmit:

```

// Add to imports:
import { digestContent } from '../src/llm/digest';
import { generateQuestions } from '../src/llm/questions';
import { initDatabase } from '../src/hooks/useDatabase';

// Replace handleSubmit with:
const handleSubmit = async (input: string) => {
  if (!apiClient) {
    Alert.alert('API Key Required', 'Please configure your DeepSeek API key in Settings first.');
```

router.push('/settings');

return;

}

setIsProcessing(true);

try {

const repo = await initDatabase();

const digest = await digestContent(apiClient, input);

// Generate a content ID

const contentId = `content\_\${Date.now()}\_\${Math.random().toString(36).slice(2, 8)}`;

await repo.insertContent({

id: contentId,

sourceType: input.startsWith('http') ? 'url' as const : 'text' as const,

title: digest.title,

rawText: input,

createdAt: Date.now(),

updatedAt: Date.now(),

});

const questions = await generateQuestions(apiClient, digest.keyConcepts, digest.title);

for (const q of questions) {

const questionId = `q\_\${Date.now()}\_\${Math.random().toString(36).slice(2, 8)}`;

await repo.insertQuestion({

id: questionId,

contentId,

type: q.type,

question: q.question,

correctAnswer: q.correctAnswer,

options: q.options,

explanation: q.explanation,

createdAt: Date.now(),

});

// Create initial SM-2 card (due immediately for first review)

await repo.upsertCard({

questionId,

easiness: 2.5,

interval: 0,

repetitions: 0,

nextReviewAt: Date.now(),

lastReviewAt: 0,

});

}

Alert.alert(

'Success!',

`Digested "\${digest.title}"\nGenerated \${questions.length} questions.\nThey're ready for review now!`,

[{ text: 'Great!', onPress: () => router.back() }],

);

} catch (error) {

Alert.alert(

'Error',

`Failed to process content: \${error instanceof Error ? error.message : 'Unknown error'}`,

);

```
} finally {
```

```
setIsProcessing(false);
```

}

};

☐ **Step 5: Update review.tsx to load due cards from database**

Update app/review.tsx — modify the useEffect:

```
// Add to imports:
import { initDatabase } from '../src/hooks/useDatabase';
import type { Question, Card } from '../src/types';

// Replace the useEffect:
useEffect(() => {
  loadDueCards();
  async function loadDueCards() {
    try {
      const repo = await initDatabase();
      const due = await repo.getDueCards(Date.now());
      if (due.length > 0) {
        startSession(due);
      }
    } catch (e) {
      console.error('Failed to load due cards:', e);
    } finally {
      setLoading(false);
    }
  }
}, []);

// Update handleAnswer to persist results:
const handleAnswer = (correct: boolean) => {
  submitGrade(correct);
  // After submitting, persist updated card + review record to DB
  // This runs after state update via a microtask
  setTimeout(async () => {
    // We access the updated card from the hook's internal state
    // For simplicity, this will be refactored in a follow-up
  }, 100);
};
```

☐ **Step 6: Commit**

```
git add app/index.tsx src/hooks/useDatabase.ts src/hooks/useSchedule.ts app/add.tsx app/review.tsx
git commit -m "feat: add home screen, database integration, and schedule hooks"
```

Task 11: Notifications + Polish

Files:

- Create: src/hooks/useNotifications.ts
- Modify: app/_layout.tsx (init notifications)

☐ **Step 1: Write notifications hook**

File: src/hooks/useNotifications.ts

```

import * as Notifications from 'expo-notifications';
import { useEffect } from 'react';

// Configure notification handler
Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowAlert: true,
    shouldPlaySound: false,
    shouldSetBadge: false,
  }),
});

export async function requestNotificationPermission(): Promise<boolean> {
  const { status } = await Notifications.requestPermissionsAsync();
  return status === 'granted';
}

export async function scheduleDailyReview(
  dueCount: number,
  hour: number = 9,
  minute: number = 0,
): Promise<void> {
  // Cancel existing scheduled notifications
  await Notifications.cancelAllScheduledNotificationsAsync();

  if (dueCount === 0) return; // Don't schedule if nothing due

  await Notifications.scheduleNotificationAsync({
    content: {
      title: 'StickyMem Review',
      body: `You have ${dueCount} question${dueCount > 1 ? 's' : ''} waiting for review.`,
      data: { screen: 'review' },
    },
    trigger: {
      type: Notifications.SchedulableTriggerInputTypes.DAILY,
      hour,
      minute,
    },
  });
}

export async function cancelNotifications(): Promise<void> {
  await Notifications.cancelAllScheduledNotificationsAsync();
}

export function useNotifications() {
  useEffect(() => {
    requestNotificationPermission();
  }, []);

  return { scheduleDailyReview, cancelNotifications };
}

```

☐ Step 2: Update `_layout.tsx` to init notifications

Modify `app/_layout.tsx`:

```

import { Stack } from 'expo-router';
import { StatusBar } from 'expo-status-bar';
import { useEffect } from 'react';
import { useDatabase } from '../src/hooks/useDatabase';
import { useNotifications, scheduleDailyReview } from '../src/hooks/useNotifications';

export default function RootLayout() {
  const repo = useDatabase();

  useEffect(() => {
    // Request notification permission on launch
    useNotifications();
  }, []);

  // Schedule daily review when due count changes
  useEffect(() => {
    if (!repo) return;
    (async () => {
      const due = await repo.getDueCards(Date.now());
      if (due.length > 0) {
        await scheduleDailyReview(due.length);
      }
    })();
  }, [repo]);

  return (
    <>
      <StatusBar style="dark" />
      <Stack
        screenOptions={{
          headerStyle: { backgroundColor: '#fff' },
          headerTintColor: '#333',
          headerTitleStyle: { fontWeight: '600' },
        }}
      >
        <Stack.Screen name="index" options={{ title: 'Today', headerShown: false }} />
        <Stack.Screen name="add" options={{ title: 'Add Content', presentation: 'modal' }} />
        <Stack.Screen name="review" options={{ title: 'Review', headerBackVisible: false }} />
        <Stack.Screen name="progress" options={{ title: 'Progress' }} />
        <Stack.Screen name="settings" options={{ title: 'Settings' }} />
      </Stack>
    </>
  );
}

```

☐ Step 3: Commit

```

git add src/hooks/useNotifications.ts app/_layout.tsx
git commit -m "feat: add daily review notifications"

```

Verification Checklist

After all tasks complete:

- ☐ npx expo start launches without errors
- ☐ Home screen shows "No content yet" empty state
- ☐ Settings screen accepts DeepSeek API key
- ☐ Adding text content triggers DeepSeek API, creates questions
- ☐ Questions appear in review queue

- ☐ Answering questions updates SM-2 schedule
- ☐ Progress screen shows accuracy data
- ☐ Daily notification triggers at 9 AM
- ☐ All tests pass: `npm test`
- ☐ App builds for web: `npx expo export -p web`
- ☐ GitHub repo is public with README

